# Checking or Recomputing? Two Cost Regimes in Verifying AI-Generated Formal Artifacts

Anonymous research draft

## Abstract

A formal proof can be cheap to check after an expensive search, yet a proof that delegates its justification to computation can make the kernel repeat that computation. We study this distinction in a released AI-generated Lean library and use it to guide ProofServe, a tool for independently rechecking compiled Lean projects. A probability sample of the pinned OpenAI mathematics library currently contains 41 of 42 original build costs. The 20 complete strata give a conditional compiler-work estimate of 278.09 CPU-hours, with a fragile nominal 95% interval of 94.15–462.04 hours; the global cost split remains uncertain. Selected complete mathematical closures require 6.31% and 8.95% of their compiler CPU for checking, whereas computation embedded in proofs remains work for the checker. ProofServe combines original-kernel declaration checking, dependency-aware parallelism, restartable segments and machine-readable coverage and axiom receipts. Its mechanisms have precedents; the systems contribution is their usable integration and evaluation. This draft separates established measurement evidence from the pending release-tool evaluation, which will determine its speed and external-project acceptance.

## 1 Introduction

Lean's kernel validates elaborated proof terms rather than repeating the tactic search that produced them. This separation motivates independent checking of AI-generated artifacts. It does not make every check cheap: definitional equality can evaluate a program embedded in a proof. At the project boundary, a user must also establish which modules were checked, which imports were trusted, and which axioms the accepted statements use.

The Lean FRO calls for routine external checking. This is already more than a roadmap: Lean 4.35 ships `lake check`, which exports and rechecks default targets, and can invoke bundled independent kernels [8, 7]. ProofServe addresses the operational cost and scope of such a workflow, rather than claiming to invent independent checking or to replace an independently implemented kernel. Its intended user supplies a Lake project and receives both a checker verdict and an explanation of its coverage.

Our empirical contribution is a pinned, reusable measurement package that distinguishes source compilation, independent replay and computation. Our systems contribution is ProofServe's project-facing check and audit interfaces, together with a fixed evaluation of the release-form tool. The latter is in progress; prototype speedups do not establish release performance. The paper retains four questions:

**RQ1** Where do verification costs fall, and how prevalent are the computation-replaying and mathematical-proof-checking regimes across the library?

**RQ2** Why does independent verification fail to realize the potential low cost of mathematical-proof checking?

**RQ3** Can computational checking scale across cores or a cluster without expanding the trusted base?

**RQ4** What gaps remain between a reported verified result and build coverage, reproducible outcomes and known axiom dependencies?

## 2 Method and workload

The primary workload is the published OpenAI mathematics repository [10], pinned at `4eaee875`; the data package records the full revision. It contains 163,097 source modules and uses Lean 4.34.1 with Mathlib `d13f23b7`. The sample has two original probability draws in each of 21 strata, combining source-size percentile, source-import depth and an explicit kernel-decision marker. The seed and all original identities are fixed. Prerequisite modules and purposefully selected expensive examples do not acquire probability-sample weights.

Source cost is native per-module compilation, including imports, tactics, kernel checking and artifact emission; prerequisite preparation, queueing, separate C compilation and Lake driver work are excluded. Checker clocks include their declared import, restore and export lifecycle. CPU is user plus system time, reported separately from wall time. Neither summed native wall intervals nor CPU divided by worker count is project-build makespan.

Timed runs use the existing exclusive P-core lease on CPUs 0–7 of an i9-14900K; untimed preparation uses

E cores. Current Lean invocations omit `-M`, including `-M0`. External charged-memory protection and sampled RSS are reported separately. Our earlier use of Lean's nondefault in-process memory checking and our prototype's missing persistence marking were measurement-method defects, not inherent workload properties. Host-reboot-interrupted attempts are retained but excluded from cost, censoring and completed samples. Completed records are integrity-checked before reuse. Repeat order and retained caches limit causal and cross-machine interpretation.

All measurement values below come from `data/openai-math-4eaee875/`. Its small tables retain versions, sample membership, missingness, clocks and estimator scope. Historical prototype interventions appear only when explaining a cause. Release system results must instead identify a committed executable under `checker/results/`; external-project results belong under `data/external-projects/`. We never substitute one evidence class for the other.

# 3 Two costs, uncertain population weights

## 3.1 A proof term can contain substantial computation

Ordinary mathematical proofs often store the result of expensive proof search as a term whose validation is much cheaper. Reflection and kernel-evaluated decision procedures can instead encode a calculation that the kernel must perform to justify the proposition [2, 5]. A short source proof is therefore not a small checking workload. Conversely, an expensive source tactic need not leave an expensive term. This stage distinction, rather than authorship or source length alone, defines the two cost regimes studied here.

The sample's static marker is any explicit comment/string-masked `decide +kernel` occurrence. Its label is a reproducible source proxy, not a complete semantic classifier: ordinary `decide`, definitional reduction and generated tactics can also perform computation. Absence of the marker does not establish inexpensive checking. A count of lexical sites cannot be multiplied by a compiler-time fraction to obtain kernel work.

## 3.2 Coverage does not imply precision

At this snapshot, 41 of 42 original source costs have completed. One ordinary draw remains unmeasured. Twenty complete strata cover 163,057 modules; the remaining 40-module stratum has only one observed original draw and is excluded from complete-cell inference. All nine marked strata are complete, covering 5,848 modules. The extra fixed precision panels remain independent of the original sample and do not enter the figures in this draft.

For complete strata, the stratified total is

$$\widehat{T} = \sum_h N_h \bar{y}_h, \qquad \widehat{V} = \sum_h N_h^2 (1 - n_h/N_h) s_h^2 / n_h.$$

The covered-domain compiler estimate is 278.09 CPU-hours: 57.55 in marked modules and 220.55 in other modules. Thus the conditional marked share is 20.69%, despite the much smaller marked-file population. This is compiler work in marker-defined files, not the fraction of CPU spent on computation inside proofs. It does not answer the full-library runtime-weight question.

The nominal 95% Student-$t$ interval is 94.15–462.04 CPU-hours, using Satterthwaite degrees of freedom 2.61. Two observations per cell and heavy tails make the approximation fragile. It excludes the incomplete stratum, nonresponse, runtime drift and transfer-model uncertainty. Its width is a substantive result: nearly complete module coverage does not make a two-draw-per-cell total precise.

A separate missing-cost scenario preserves all known original costs and weights, assigning the last draw the observed ordinary-class minimum or maximum. It gives 278.5–279.1 CPU-hours after outward rounding. This narrow range concerns completion of the fixed sample, not its sampling error. Without assumptions on the missing cost, the completion-cost upper remains unbounded. Checker-transfer scenarios are models, not measured whole-library savings.

**Design implication.** An audit should expose the source proxy, measured coverage and interval separately. It should never turn an unknown module into a zero-cost module or a marker into a dynamic phase verdict. ProofServe therefore reports actual checked scope and accepts costs only as explicitly supplied observations.

# 4 Why cheap checking is hard to realize

## 4.1 Source production and independent replay have different work

The complete selected closures in Table 1 show that checking can be substantially cheaper than producing a mathematical artifact. The checkpointed Falconer closure uses 2,038.44 CPU-seconds for checking against 32,292.60 for compilation, or 6.31%. A Mathlib closure under the same checkpoint protocol uses 91.15 against 1,018.22 CPU-seconds, or 8.95%. Their wall ratios are 9.26% and 14.67%, respectively. CPU and wall answer different questions and are not interchangeable speedups.

Independent checking still requires producing its input artifact. The source denominator includes imports,

| Closure | Modules | CPU ratio | Wall ratio |
|---|---|---|---|
| Mathlib Nat.Basic | 1,330 | 8.95% | 14.67% |
| Falconer | 13,205 | 6.31% | 9.26% |
| Grid | 3,793 | 8.47% | 14.52% |
| Amplitudes | 5,083 | 8.44% | 14.55% |

Table 1: Measured complete-closure check/source-work ratios from the public data package. Mathlib and Falconer use checkpointed replay; Grid and Amplitudes use the older uninterrupted stream. Shared dependencies make these overlapping cases, not independent population samples. Design implication: charge checking preparation at the complete-closure boundary.

tactics and emission; replay includes reading and its declared lifecycle. Their ratio is neither a kernel fraction nor an interactive-service or project-rebuild speedup.

Whole-closure averaging also hides where the work sits. A closure includes many reused mathematical dependencies around a computation-heavy target. Grid and Amplitudes therefore do not establish that every expensive certificate is cheap to recheck. Their aggregate CPU ratios cannot be assigned to all marked modules as a measured population law. This explains why we retain selected-case ratios while refusing a global checking-cost point estimate.

### 4.2 Imports and repeated preparation consume the opportunity

Starting a checker independently for every module repeats process and import work. Importing an entire project's final environment once avoids some repetition, but it changes what is being checked. A module-local path may trust its imports; a final merged environment may hide distinctions between incoming definitions. Checking every module in its own declared dependency context is a different ledger from checking one final map of constants.

The bundled Lean 4.34.1 checker already dispatches module-local tasks [4]. Its default path loads imports and replays local declarations; `--fresh` replays the final imported environment from an empty kernel environment. In the pinned implementation the latter replay traverses that map sequentially. This observation is version-specific, not a claim that official checking has no parallelism. Target-name equality alone does not justify a performance ratio between these procedures and a dependency-ordered per-module verifier.

These boundaries motivate a reusable checked prefix, but such reuse has a trust cost. The reader, module visibility, duplicate handling and restored metadata must preserve the intended context. Unchanged mathematical kernel rules do not make the entire checker implementation unchanged. ProofServe reports its execution path and actual accepted modules so a consumer can distinguish these cases.

### 4.3 Prototype interventions identify engineering causes

Our prototype initially shared frontiers without explicit persistent marking. The corrected same-executable intervention reduced eight-worker CPU by 14.13% and wall time by 5.13% on its measured closure. This corrects our own implementation method. It does not establish that reference counting explains every residual cost or that Lean intrinsically scales poorly.

A second prototype policy replayed an entire module serially when any nonordinary declaration prevented its ordinary-declaration plan. Retaining a dependency-closed nonordinary prefix and parallelizing the remainder reduced the paired plain-run wall median from 257.66 to 196.67 seconds, or 23.67%, at a 0.106% CPU-median increase. The prefix is a conservative implementation choice, not a proved serial lower bound. The intervention's flags, version, full clocks and trust boundary are recorded in the evidence map. These causal observations do not fill the release-tool evaluation below, and their ratios are not multiplied with other experiments.

**Design implication.** Independent checking should amortize compatible preparation, retain module context and measure complete operations. Parallelism must come from actual declaration dependencies, while memory lifetime and fallback policies remain explicit implementation decisions. A faster leaf interval alone is insufficient evidence of a faster project check.

## 5 ProofServe design

ProofServe exposes `check`, `audit` and the existing Kimina-compatible `serve` interface. The service is maintained for compatibility; this work adds no serving feature. Figure 1 shows the project workflow. The native checker and audit implementation are available; packaging and release-level acceptance remain in progress.

### 5.1 Check a dependency closure

`proofserve check <project>` uses the project's declared toolchain and compiled artifacts. By default the native checker discovers compiled local roots; explicit roots support custom layouts. It traverses transitive imports and rejects missing artifacts instead of treating them as already checked. Uncompiled source files remain outside the accepted compiled closure and are exposed separately by audit.

For ordinary declarations, the checker constructs a local dependency graph from their referenced constants. Ready declarations run against a checked frontier using the original kernel's checking operation. Workers return

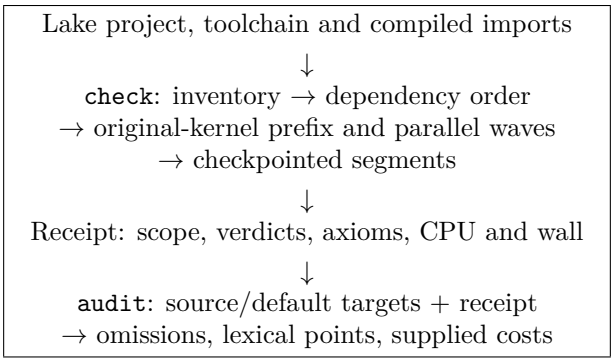



Figure 1: Separate proof acceptance from its coverage explanation. Audit consumes reported evidence; it does not independently certify the checker.

checked values; a coordinator validates their identities and commits the frontier after a wave joins. Nonordinary declarations and their ordinary prerequisites use the original replay path before the remaining ordinary graph runs. Cyclic or unsupported local structure retains a conservative fallback. No temporary axiom skeleton or repaired proof replaces the input.

Persistent marking occurs only after earlier workers join and before new ones start. The release tool exposes disabling persistence and split fallback for the fixed ablations. These flags test the integrated implementation; prototype intervention results do not determine the outcome. Worker scaling is measured in both wall and CPU; it need not be monotonic.

### 5.2 Resume without relabeling reused work

The checker processes bounded segments and serializes its successful prefix. A repeated invocation can reuse completed segments when the toolchain, artifacts, Lake lock and checkpoint configuration match. A live invocation holds its checkpoint directory lock. An interruption does not create an accepted segment; an incomplete receipt identifies only the successful prefix. Restoration of an accumulated prefix still consumes memory, so segmentation is not a guarantee that an arbitrarily large environment fits a small memory budget.

Receipt v1 records the execution path, Lean and dependency identity, module set hash, accepted modules, rejection events, axiom set and clocks. Current-invocation CPU and reused cumulative segment CPU must be distinguished. A completed checkpoint cache hit is not a fresh timed verification run. The interruption experiment charges restart and restoration overhead.

### 5.3 Audit what verified covers

`proofserve audit <project>` inventories local source, extracts imports and masks comments and strings before counting kernel-decision forms. Supported declarative Lake configuration yields a default-target source closure and omissions. Executable configuration, custom targets or unsupported constructs produce an explicit unknown result unless the caller supplies evaluated configuration or roots. Supplied roots are never relabeled as Lake's defaults.

Audit can consume receipt v1 to display the checked compiled closure, verdicts and reported axioms. An incomplete receipt establishes only the axioms of its last successful prefix. Per-module cost logs require explicit source digests; aggregate checker clocks are not redistributed into invented module costs. Missing costs stay missing. Lexical classes and measured costs remain separate fields, because neither source markers nor an accepted theorem establish its dynamic regime.

The current receipt does not bind its compiled evidence to the current source digest. Audit makes that gap explicit rather than certifying a changed source tree using an old receipt. Generated modules, external source trees and executable Lake behavior also require separate coverage evidence.

### 5.4 Trust and compatibility

ProofServe independently reruns a workflow using Lean's original kernel; it is not a separately proved or independently implemented kernel. It additionally trusts the artifact reader, dependency/context metadata, checked-token FFI adapter, quiescent persistent marking and its own locally generated checkpoints. Serialized private metadata and an internal Lean symbol require a matching SDK. Input hashes detect ordinary staleness; they do not authenticate maliciously modified checkpoints.

Axioms remain axioms. Acceptance does not mean axiom-free, absence of `sorryAx`, correct tactic plugins, or correct executable code. Upstream unsafe/partial exclusions remain visible. On an unsupported or mismatched toolchain, the command must fail explicitly or select the project's matching official checker, with that path and its narrower receipt scope identified. It must never silently decode another version's artifacts or count official fallback as a parallel-tool measurement.

## 6 Release-tool evaluation

The fixed evaluation answers the systems portions of RQ3 and RQ4. It is limited to the committed `proofserve-check` executable and `proofserve audit`. Every system row must identify the executable commit, project commit, toolchain, roots, accepted module set and timing boundary. Table 2 reserves the result layout. The available one-point record predates the frozen final matrix and is excluded. No release speedup is established in this draft.

| Closure | Workers | CPU / wall | Set |
|---|---|---|---|
| Mathlib Nat.Basic | Pending | Pending | Pending |
| Grid | Pending | Pending | Pending |
| Barrier | Pending | Pending | Pending |

Table 2: Reserved release-matrix layout: three fresh-checkpoint repeats per worker count. Cells await the frozen executable's complete results; historical prototype values are not substituted.

### 6.1 RQ3: complete scaling, ablations and official scope

Mathlib Nat.Basic, Grid and Barrier are checked at 1, 2, 4 and 8 workers, with three fresh-checkpoint processes per point. Mathlib in that matrix means the Nat.Basic closure, not the whole source tree; any separate whole-Mathlib run has its own scope and row. We report wall and CPU medians, ranges and matched acceptance sets. Each cell includes process startup, artifact reading and its checkpoint lifecycle. Queueing for the common P-core lease is outside the native clock and recorded separately.

At eight workers, Grid and Mathlib each receive three runs without persistence and three without split fallback. Other settings and inputs match. These ablations test whether prototype causes survive integration; CPU and memory accompany wall improvements.

Official `leanchecker` default and `--fresh` runs use the same resource point and complete recorded command clocks. Default name-prefix selection can include extra modules, and fresh mode can have a different declaration ledger. We compare acceptance only at a demonstrated common scope and report any unmatched work; where equivalence is absent, both costs are shown without a speedup ratio. Earlier leaf-only official timings are not an admissible full-closure denominator. The fixed comparison does not establish superiority to every current official export or independent kernel path.

RQ3's multi-core result retains additional trusted coordination code; it cannot establish an unchanged implementation TCB. Cluster checking is unevaluated and cannot be inferred from single-host curves.

### 6.2 RQ4: acceptance, rejection, resumption and external use

Development controls exercise loose bound variables, unknown constants and wrong sorts, and compare small accepted closures with the official checker. These are finite functional checks, not the completed release CI or a general soundness proof. The release test requires all three bad-proof classes to be rejected, accepted module sets and reported axioms to agree at the comparable scope, and the interruption/resume run to match its uninterrupted counterpart. Additional CPU and wall time from restart are reported as costs rather than omitted from the successful run.

| Release acceptance evidence | Current disposition |
|---|---|
| Three bad-proof classes in CI | CI closure pending |
| Interrupted vs. uninterrupted | Timed pair pending |
| Three unseen public projects | Results pending |
| At least one published AI project | Provenance pending |
| Package/container, no Workspace paths | Clean-start acceptance pending |

Table 3: External and functional acceptance obligations. Pending entries are not zero costs, failed proofs or completed user acceptance.

External validation uses at least three public Lean projects not used during development, including at least one with published AI-generated source. For each, the result records its name and immutable commit, AI provenance, module count, tool and official CPU/wall costs, acceptance parity, audit findings and encountered problems. A checker fixture alone does not establish AI provenance. Clean-start package/container execution must work without Workspace paths.

Audit acceptance examines both known and unknown coverage: default roots that omit local modules, unsupported executable configuration, source changes after a receipt, incomplete-prefix axiom reporting and supplied cost-log mismatch. It must preserve explicit zero counts while distinguishing an incomplete scan from an empty finding. These are user-visible coverage findings for RQ4.

**Artifact and reproduction.** The artifact is at https://github.com/yunwei37/proofserve. The intended installed interface is `proofserve check /path/to/project` followed by `proofserve audit /path/to/project`. The current native entry point is `proofserve-check`, built by `cd checker; lake build`; audit also has the portable entry point `python3 tools/audit/audit.py`. The data-only command `python3 data/openai-math-4eaee875/summarize.py` recomputes the published sample summary without Lean or raw Workspace captures. Release installation, complete CI and external results remain acceptance requirements, not claims made by these commands.

## 7 Limitations and related work

The current sample is incomplete, its static labels are proxies, and its nominal sampling interval is wide. Closure ratios cover selected, overlapping cases and retain their original protocol differences. They do not identify global kernel phase weights or the checking cost of the entire AI library. Release evaluation and external acceptance are unfinished; their reserved cells will be filled

only from the designated committed tools. Hardware, cache order, observers and source/compiled binding limit transfer beyond the measured environment.

Parallel proof checking, asynchronous tasks and certified decomposition have substantial precedent in Isabelle, Coq and Lean [12, 1, 3]. Existing Lean checkers and the shipped Lake workflow address trust and rechecking [6, 7]; ProofServe does not claim mechanism firstness. Kimina and AXLE already provide verification services [11, 13], so a new serving feature is not a contribution here. A public technical report also studies build/check costs and computation in the same OpenAI library [9]; this paper claims neither the first same-corpus characterization nor the first observation that checking differs from recomputation.

# 8 Conclusion

Proof checking and proof-embedded recomputation create different opportunities, while preparation and coverage determine whether an apparent saving survives at the project boundary. The pinned data package establishes selected cheap-checking cases and a population estimate whose uncertainty remains large. ProofServe turns those lessons into an explicit check/audit workflow with original-kernel checking and declared additional trust. Release scaling, correctness and external acceptance remain the test of its practical value.

# References


[1] Bruno Barras, Carst Tankink, and Enrico Tassi. Asynchronous processing of coq documents: from the kernel up to the user interface, 2015.

[2] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction (CADE 28)*, pages 625–635, 2021.

[3] Lean contributors. Lean 4.19.0: Parallel elaboration, 2025.

[4] Lean contributors. Lean 4.34.1 bundled checker: module task dispatch. https://github.com/leanprover/lean4/blob/v4.34.1/src/LeanChecker.lean, 2026.

[5] Lean contributors. Lean 4.34.1 decision-tactic implementation, 2026.

[6] Lean contributors. lean4checker and the built-in leanchecker, 2026.

[7] Lean FRO contributors. Lean 4.35.0 release notes: Lake checking and comparator, 2026.

[8] Lean FRO contributors. The lean fro year 4, part 1 roadmap, 2026.

[9] nealmcb. Verifying openai/math in lean: cost and checker study, 2026.

[10] OpenAI math contributors. math: Mathematical manuscripts and supporting proof artifacts, 2026. Pinned source release 4eaee87548250b2682affda7d05d947c05554bae; accessed 30 October 2026.

[11] Marco Dos Santos, Hugues de Saxcé, Haiming Wang, Ran Wang, Mantas Bakšys, Mert Ünsal, Junqi Liu, Zhengying Liu, and Jia Li. Kimina lean server: A high-performance lean server for large-scale verification. In *Fifth Workshop on Mathematical Reasoning and AI (MATH-AI) at NeurIPS*, 2025.

[12] Makarius Wenzel. Parallel proof checking in isabelle/isar. In *ACM SIGSAM Workshop on Programming Languages for Mechanized Mathematics Systems (PLMMS)*, 2009.

[13] Jimmy Xin, Alex Schneidman, Chris Cummins, Karun Ram, Srihari Ganesh, and Jannis Limperg. AXLE: A cloud infrastructure for lean 4 theorem proving utilities, 2026. Version 1; accepted at the 3rd AI for Math Workshop, ICML 2026.